NutriTrackWeb

Technical Documentation

NutriTrackWeb – Web application for nutrition counselling

Name	Stefany Castelão dos Santos
Class	3. IT INF – Labor
Group	3APP1
Date	08.11.2026
Teacher	Herr Platzer
Project	NutriTrackWeb

Table of Contents

1 Overview and Architecture 3
1.1 Components 3
1.2 Flow of a Request 3
1.3 Roles 4
2 Glossary 4
3 Project Structure 5
3.1 Files and Folders 5
3.2 CSS and JavaScript 6
4 Database 6
4.1 Tables 7
4.2 Third Normal Form 8
4.3 Status Codes 8
5 Security and Input Validation 9
5.1 Measures at a Glance 9
5.2 Input Validation 10
5.3 Uploads and Protected Folders 10
5.4 Test Records for XSS Protection 10
6 Installation and Start 10
6.1 Steps 10
6.2 Setting in php.ini 11
6.3 Local HTTPS (mkcert) 11
Certificate 12
6.4 Demo Accounts 13
7 Limits and Extensions 14
7.1 Known Limits of Version 1 14
7.2 Later Extensions 14

1 Overview and Architecture

NutriTrackWeb is a PHP web application in which a nutritionist manages her clients. The PHP pages talk to a MariaDB database through PDO and deliver finished HTML to the browser. The technical terms are explained in chapter 2.

1.1 Components

Component	Used	Purpose
Language	PHP, no framework (one file per page)	Logic and output of the pages
Database	MariaDB, database *mydb*	Stores users, clients, appointments, messages, payments, etc.
Data access	PDO with prepared statements	Secure access to the database
Web server	Apache (XAMPP, Windows) with HTTPS	Serves the pages
Interface	HTML, CSS, some JavaScript	Display; colours and logo can be changed in the settings
Languages	English, German, Portuguese	The texts are stored in the array *$t* in every PHP file

1.2 Flow of a Request

1. The browser opens a PHP page, for example *client_home.php*.
2. The page loads *config.php*: it starts the session (30-minute timeout) and creates the database connection *$pdo*.
3. The page checks whether someone is logged in and which role they have (*users.role*: 0 = User, 1 = Admin). Without the right role, the user is sent back to the login.
4. For forms (POST), the CSRF token is checked first, then the input is validated.
5. The page reads or writes data with prepared statements. Writes that belong together run in one transaction (*rollBack* on error).
6. After saving, the page redirects (PRG pattern), so reloading the page does not save anything twice.
7. All output goes through *e()* (= *htmlspecialchars*), so no foreign code runs in the browser.

1.3 Roles

Role	Can	Cannot
Admin (nutritionist)	Review registrations and manage clients, appointments, documents, meal photos, messages, payments and settings	–
User (client)	See and edit their own data, appointments, documents, meal photos, messages and payments	Open Admin pages; see other people's data or files

Clients register themselves. Access only applies after approval by the Admin (*clients.approval_status*: 0 Pending, 1 Approved, 2 Rejected, 3 MoreInfos).

2 Glossary

The most important terms, explained simply:

Term	Simple explanation
Web server (Apache)	The program that delivers the pages when the browser asks for them. Like a waiter who takes the order and brings the food.
PHP	A programming language that runs on the server. PHP builds the page, and the browser only receives the finished HTML.
XAMPP	A package for your own computer with Apache, MariaDB, PHP and phpMyAdmin in one installation.
MariaDB	The database server: the “filing cabinet” where all data is stored in tables. It is a free continuation of MySQL; the “MySQL” in XAMPP is actually MariaDB.
SQL	The language used to read and write data in the database (for example *SELECT*, *INSERT*, *UPDATE*).
PDO	PHP Data Objects: PHP's standard way to talk to a database. Like a universal plug that offers the safe route with prepared statements.
Prepared statement	An SQL command with placeholders (*?*). The values are sent separately. Like a form with blanks: what is written in a blank cannot change the text of the form.
Session	A note on the server for each visitor. The browser only carries a number (like a cloakroom ticket); the data stays on the server.
Hash	A kind of fingerprint of a password. It can be calculated, but cannot be turned back into the password.

Term	Simple explanation
CSRF	A foreign website tries to send a form to our site in your name. Protection: a secret token in the form.
XSS	Someone writes program code (for example *<script>*) into a text field so that it runs in other people's browsers. Protection: output is displayed as plain text.
SQL injection	Someone writes SQL commands into a field so that the database executes them. Protection: prepared statements.
Transaction	A group of write operations that only count together: all or nothing.
.htaccess	A small configuration file for Apache. Here it blocks direct access to confidential uploads and prevents program code in the upload folders.
HTTPS and certificate	HTTPS encrypts the connection between browser and server. The certificate is the server's “ID card”, so the browser trusts it.
mkcert	A tool that creates a certificate for your own computer, which the browser on this computer trusts.
phpMyAdmin	A web interface to view, import and export the database.

3 Project Structure

3.1 Files and Folders

The project is located in *C:\xampp\htdocs\nutritrackweb*. All 24 PHP files are in the main folder. In addition there is the folder *icons/* (SVG icons) and the folder *uploads/* (uploaded files).

Area	Files	Purpose
Base	*config.php*	Session start, timeout, database connection (PDO)
Access	*index.php*, *login.php*, *logout.php*, *register.php*	Start page, login, logout, self-registration
Admin	*admin_dashboard.php*, *registration_review.php*, *clients.php*, *appointments.php*, *documents.php*, *meal_photos.php*, *messages.php*, *payments.php*, *settings.php*	Dashboard, review registrations, clients, appointments, documents, meal photos, messages, payments, settings

Area	Files	Purpose
User	*client_home.php*, *client_status.php*, *client_profile.php*, *client_appointments.php*, *client_documents.php*, *client_meal_photos.php*, *client_messages.php*, *client_payments.php*	Home, approval status, profile, appointments, documents, meal photos, messages, payments
File access	*download.php*, *view_image.php*	Only way to reach uploaded files, with permission check
Folders	*icons/*, *uploads/documents/*, *uploads/meal_photos/*, *uploads/profile/*	Icons; protected upload folders

Every page has the same structure: the PHP logic at the top (check role, process POST, load data), the HTML below it. The texts for English, German and Portuguese are at the beginning of each file in the array *$t*. The source code is commented in German throughout.

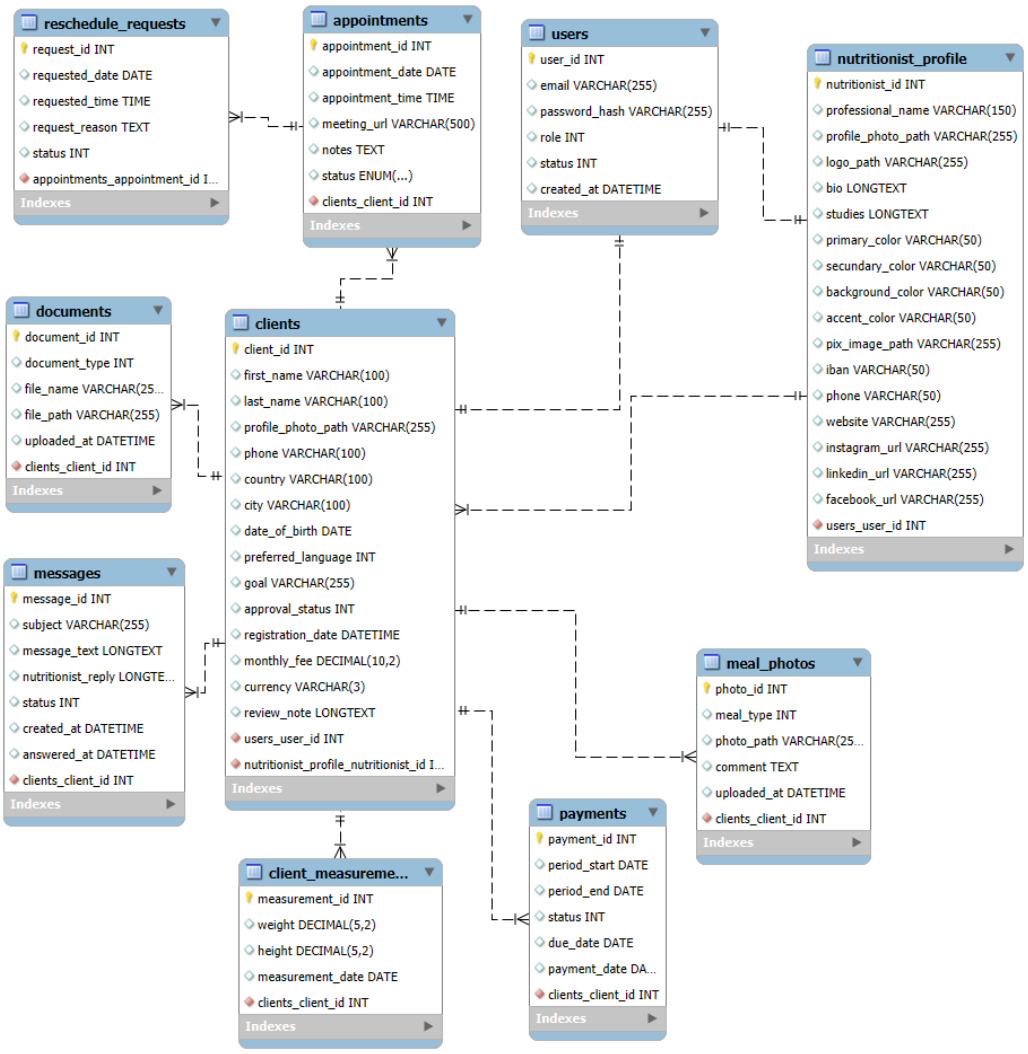
3.2 CSS and JavaScript

CSS and JavaScript are not in separate files but inside each PHP page: the CSS in a *<style>* block at the top of the file, the JavaScript in a *<script>* block at the end. For the CSS there is a reason: the colours come from the database (table *nutritionist_profile*). PHP writes them into the block as CSS variables (*--primary*, *--accent*, etc.) when the page loads. This way, the look of the whole application changes as soon as the Admin chooses other colours in the settings.

> **Note:** Separate files (*style.css*, *app.js*) would be the more professional solution, because identical rules would be stored only once and the browser could cache the files. This is planned for a later version (see chapter 7).

4 Database

The database *mydb* has 10 tables (InnoDB, utf8) and satisfies the third normal form. The complete export is in *nutritrackweb_database_dump.sql*.



ER diagram of the database mydb (MySQL Workbench)

4.1 Tables

Table	Content	Link (foreign key)
users	Login: e-mail, *password_hash*, role, status	–
nutritionist_profile	Name, bio, colours, logo, IBAN, contact links of the nutritionist	*users_user_id*
clients	Master data, goal, approval status, monthly fee, currency	*users_user_id*, *nutritionist_profile_nutritionist_id*
client_measurements	Weight, height, date	*clients_client_id*
appointments	Date, time, meeting link, notes, status	*clients_client_id*

Table	Content	Link (foreign key)
reschedule_requests	Rescheduling: requested date, reason, status	*appointments_appointment_id*
documents	PDF files (name, path, type, date)	*clients_client_id*
meal_photos	Meal photos with comment	*clients_client_id*
messages	Message from the client and reply from the nutritionist	*clients_client_id*
payments	Billing period, due date, payment date, status	*clients_client_id*

4.2 Third Normal Form

- Every table has exactly one single-column primary key (for example *client_id*).
- Every piece of information is stored only once: the amount of a payment is not stored twice, it comes from *clients.monthly_fee*.
- Country and city are free text fields and not separate tables, because the application only displays them.
- Status values are numeric codes that are explained in the PHP code (see 4.3).

Reading and writing always go through PDO on every page. Writes across several tables (for example the registration with *users*, *clients* and blood test in *documents*) run in one transaction: either everything is saved or nothing.

4.3 Status Codes

Field	Values
users.role	0 User, 1 Admin
clients.approval_status	0 Pending, 1 Approved, 2 Rejected, 3 MoreInfos
messages.status	0 New, 1 Answered, 2 Closed
payments.status	0 Open, 1 Paid, 2 Cancelled
reschedule_requests.status	0 Pending, 1 Approved, 2 Rejected
appointments.status	scheduled, rescheduled, completed, cancelled (text)

5 Security and Input Validation

5.1 Measures at a Glance

Threat	Measure	Simply explained
Readable passwords	*password_hash()* when saving, *password_verify()* at login (*register.php*, *login.php*). Only the hash is stored in *users.password_hash*. Minimum length 8 characters.	The password is never stored, only a “fingerprint”. At login, the fingerprint of the input is compared. If the database is stolen, the passwords stay unreadable.
Stolen session	*session_regenerate_id(true)* after login. After 30 minutes without activity, *session_unset()* and *session_destroy()* (all pages).	After login there is a new “cloakroom ticket”; an old ticket is worthless. After 30 minutes of inactivity the ticket becomes invalid.
Forged forms (CSRF)	Random token (*random_bytes*) in session and form, compared with *hash_equals()* (all pages with POST).	Every form contains a secret token that only our site knows. If it does not match, the form is rejected.
Foreign code in the output (XSS)	Every output goes through *e()* = *htmlspecialchars()* (all pages).	Whoever writes <*script*> into a field only sees it as normal text afterwards. It does not run as a program.
SQL injection	Only prepared statements with *?* placeholders (all pages).	The command is sent first as a template, the values afterwards and separately. The database treats input only as data, never as a command.
Wrong role, other people's data	Every page checks *users.role*. Users only see rows with their own *client_id*.	A “doorman” on every page: a User cannot reach Admin pages or other people's data.
Half-saved data	Transaction with *rollBack* (*register.php*, *appointments.php*, *client_status.php*, *registration_review.php*).	All or nothing: on an error, everything is undone.
Double submission	Redirect after saving (PRG pattern, all forms).	After saving, the page reloads. F5 does not send the form again.
Bypassed browser validation	Validation also in PHP, not only with *required* in the browser.	Browser validation can be bypassed. That is why the server validates too.
Intercepted connection	HTTPS with certificate (see 6.3).	The data between browser and server is encrypted.

5.2 Input Validation

The checks always run on the server; in the browser they exist only in addition (for example *required*). Examples from *register.php*:

- Required fields must not be empty.
- E-mail: *filter_var(..., FILTER_VALIDATE_EMAIL)* and at most 255 characters.
- Password: at least 8 characters.
- Goal: at most 255 characters.
- Privacy policy: the checkbox is also checked in PHP (*isset($_POST['privacy'])*).

Errors appear as a message in the selected language. The values already entered stay in the form.

5.3 Uploads and Protected Folders

- The folders *uploads/documents* and *uploads/meal_photos* contain a *.htaccess* with *Require all denied*. The browser cannot open any file there directly.
- The folder *uploads/profile* (profile photos, logo, PIX QR code) has a *.htaccess* that blocks program files (for example PHP) and hides the file list. The images themselves are displayed directly, because they are not confidential (for example the photo on the login page).
- Files are only delivered through *download.php* and *view_image.php*. Both check login and permission. For other people's or missing files, the answer is always the same *404*, so nobody can find out whether a file exists.
- File names are assigned randomly. Only the *basename()* inside the fixed folder is delivered (*realpath*).
- The content is checked, not only the extension: a PDF starts with *%PDF-* (at most 10 MB); images are checked with *getimagesize()* (at most 8 MB).

5.4 Test Records for XSS Protection

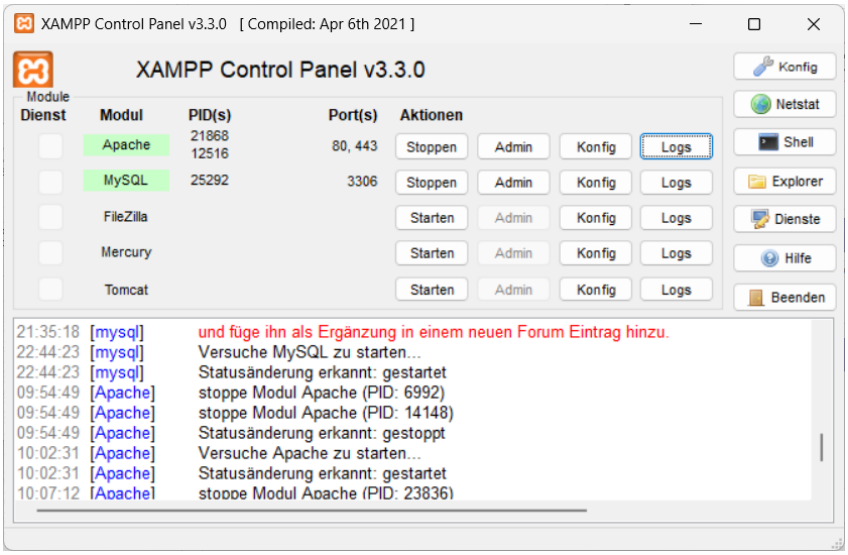
The demo database deliberately contains a client named *Teste* and a message containing *<script>alert(1)</script>*. The application displays this text unchanged, without executing it. This is the proof that the protection against XSS (*e()*) works.

6 Installation and Start

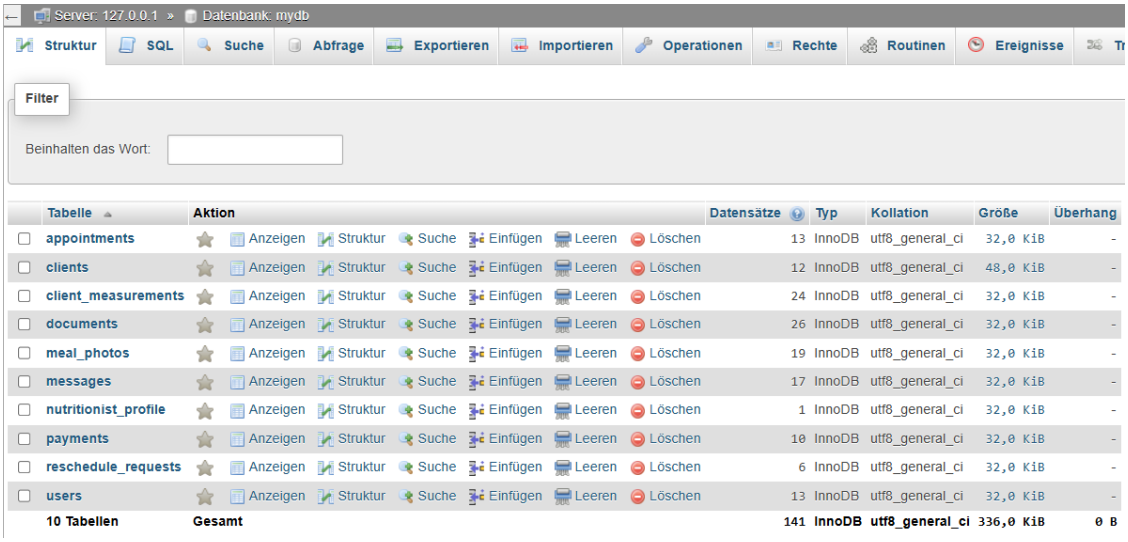
6.1 Steps

1. Install XAMPP and start **Apache** and **MySQL** (MariaDB) in the Control Panel.
2. Copy the project folder to *C:\xampp\htdocs\nutritrackweb*. The folder *uploads/* must contain the subfolders *documents*, *meal_photos* and *profile*. Each of them holds a file *.htaccess*.

3. In phpMyAdmin (*http://localhost/phpmyadmin*), create the database *mydb* and import *nutritrackweb_database_dump.sql*.
4. Check in *config.php*: database *mydb*, user *castelao*, empty password (for local development only).
5. Open *https://localhost/nutritrackweb/* in the browser and log in with a demo account (see 6.4).



XAMPP Control Panel with Apache and MySQL running



phpMyAdmin: database mydb with its 10 tables

6.2 Setting in php.ini

upload_max_filesize and *post_max_size* must be at least *10M*, otherwise uploading PDFs fails. In this XAMPP installation both are set to *40M*. After a change, restart Apache.

6.3 Local HTTPS (mkcert)

So that *https://localhost* works without a browser warning, the tool mkcert creates a locally trusted certificate.

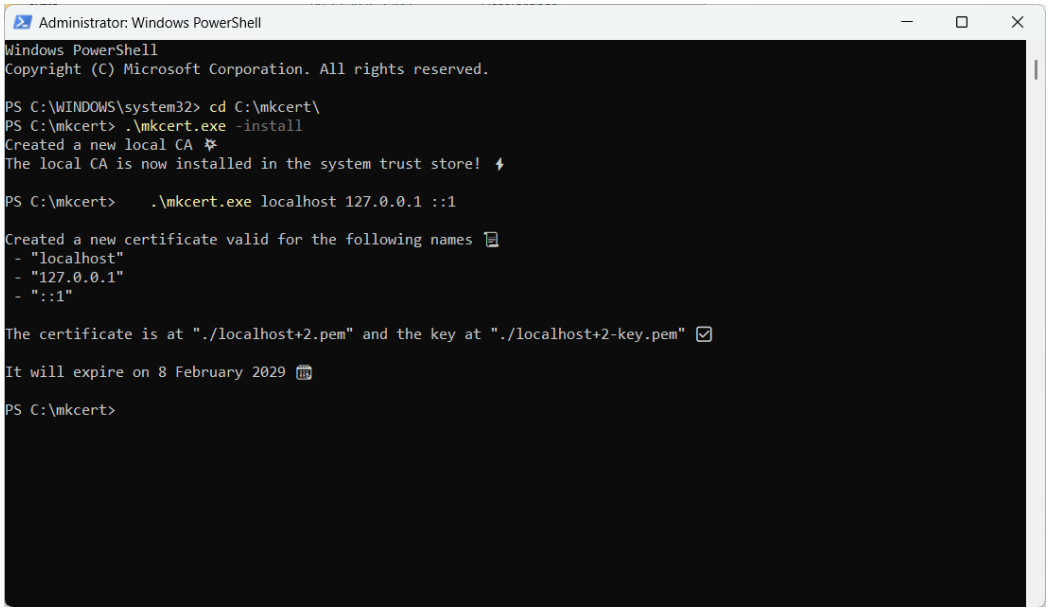
1. Download mkcert from the project's GitHub page (“Releases” page, file for Windows, *mkcert-…-windows-amd64.exe*). Put the file into the folder *C:\mkcert* and rename it to *mkcert.exe*.
2. Open **Windows PowerShell as administrator** and install the local certificate authority:

```
cd C:\mkcert\
.\mkcert.exe -install
```

3. Create the certificate for *localhost*:

```
.\mkcert.exe localhost 127.0.0.1 ::1
```

mkcert creates the files *localhost+2.pem* (certificate) and *localhost+2-key.pem* (key).



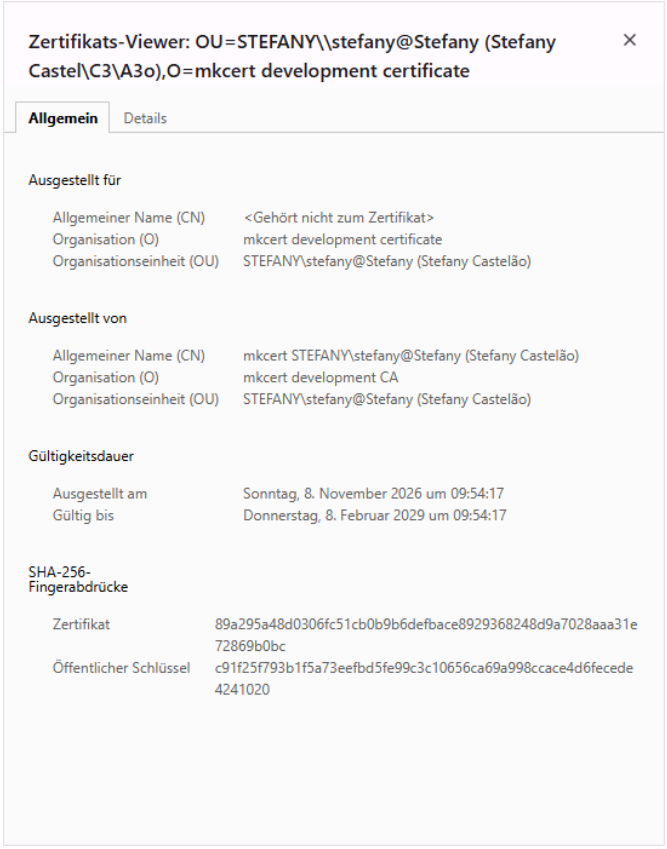
PowerShell: install the local certificate authority and create the certificate

4. Copy both files into the Apache folder of XAMPP and enter them in *C:\xampp\apache\conf\extra\httpd-ssl.conf* at *SSLCertificateFile* and *SSLCertificateKeyFile*.
5. Restart Apache in the XAMPP Control Panel and open *https://localhost/nutritrackweb/*.

Certificate

Item	Value
Issued by	mkcert development CA
Issued to	mkcert development certificate (names: *localhost*, *127.0.0.1*, *::1*)
Valid from	08.11.2026, 09:54
Valid until	08.02.2029, 09:54

Item	Value
Files	*localhost+2.pem*, *localhost+2-key.pem*



Certificate viewer in the browser showing the validity period

Important: The certificate is only trusted on the computer where *mkcert -install* was run. On another computer the browser shows a warning. The presentation is therefore given from the author's own notebook.

6.4 Demo Accounts

The database export contains demo accounts with invented data. The passwords are in the file *Zugangsdaten.txt* in the submission ZIP. The status of an account (approved, pending, rejected) may change during testing.

Name	E-mail	Role
Marianne Böhm, MSc	admin-demo@ntw.at	Admin
Lucas Pereira	user1-demo@ntw.at	User
Anna Fischer	user2-demo@ntw.at	User
Emily Carter	user3-demo@ntw.at	User
Lukas Schneider	user4-demo@ntw.at	User

Name	E-mail	Role
Beatriz Oliveira	user5-demo@ntw.at	User
Leonardo Barbacoa	barbacoa@gmx.at	User
Roger Batista	user7-demo@ntw.at	User
Amanda Moura	user8-demo@ntw.at	User
Sophie Gruber	user9-demo@ntw.at	User
Markus Steiner	user10-demo@ntw.at	User
Teste (XSS test)	user11-demo@ntw.at	User
Amanda Silva	user12-demo@ntw.at	User

7 Limits and Extensions

7.1 Known Limits of Version 1

- The application only runs locally. Hosting on the internet was not implemented, even though a ticket exists for it (see 7.2).
- The database access has no password, and HTTPS only works with the local mkcert certificate. This is meant for development and presentation, not for real operation.
- CSS and JavaScript are stored separately in every PHP file (see 3.2). Country and city are free text, and the interface texts are stored in each file instead of central language files.
- Clients cannot be deactivated and reactivated (unused code was removed).
- There are no automated tests. Testing was done manually with a test script of 132 points.

7.2 Later Extensions

- **Hosting:** publication on the internet with its own domain, a real certificate (Let's Encrypt) and a database user with a strong password.
- Move **CSS and JavaScript** into separate files (*style.css*, *app.js*); only the colour variables remain dynamic.
- Central language files instead of texts in every PHP file.
- Forgot password (reset by e-mail).
- Warning about poor contrast in the colour settings.
- Formal security review of the whole application.